

DATAFRAME

```
import pandas as pd
import matplotlib.pyplot as plt

# how to store data in pandas using dictionary
df = pd.DataFrame(

    {

        "name" : ["ayush", "anmol", "vishwas", "sakshi"],

        "age": [18, 19, 19, 19],

        "gender" : ["male", "male", "male", "female"],

    }

)

df
```

```
# how to access series from given dataframe

df["age"]
```

finding maximum and minimum value of any particular column directly

```
df[column_name].max()
```

```
df[column_name].min()
```

finding maximum and minimum value of all columns directly

```
df.max()
```

```
df.min()
```

```
# creating a series
ages = pd.Series([17, 16, 15, 13], name = "age")
print(ages)
```

```
# interested in numerical data stats
df.describe()
```

```
# reading a csv file
```

```
heart_disease = pd.read_csv("/content/heart_disease.csv")
# printing first five and last five rows, which is shown by default
heart_disease
```

```
# want to see first 8 rows of the csv file using head()
heart_disease.head(8)
```

```
# want to return last 8 rows of the csv file using tail()
heart_disease.tail(8)
```

```
# interpreting the datatype of each column using dtypes
heart_disease.dtypes
```

```
# to know the technical summary of a DataFrame
heart_disease.info()
```

```
# how to select and work with the subsets of DataFrame
heart_disease["age"].head()
```

```
# knowing the shape of a subset as shape is also an attribute
heart_disease["age"].shape
```

```
# selecting two series at the same time as inner bracket is used to
define python list with column names and outer bracket is used to
select data from the dataframe
```

```
heart_disease[["age", "sex", "cp"]].head()
```

```
# for finding the most common value in any particular column
```

```
# for finding frequency of each value in a particular column
```

```
df[column_name].value_counts()
```

output:

column_name	frequency
.	.
.	.

uses of loc[]

The `loc[]` method in pandas is a label-based indexing method used for selecting rows and columns in a DataFrame by label(s) or a boolean array. It allows you to access a group of rows and columns by labels or conditions. Here are different cases of using `loc[]` with examples:

1. Selecting rows by label

You can use `loc[]` to select a specific row by its index label.

```
import pandas as pd
```

```
# Create a sample DataFrame
```

```
df = pd.DataFrame({  
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],  
    'Age': [25, 30, 35, 40],  
    'City': ['New York', 'Paris', 'London', 'Berlin']  
})
```

```
# Select row by label
```

```
print(df.loc[1])
```

output:

```
Name    Bob
```

Age 30

City Paris

Name: 1, dtype: object

2. Selecting multiple rows by label

You can use a list of labels to select multiple rows.

```
# Select multiple rows by label
```

```
print(df.loc[[0, 2]])
```

output:

	Name	Age	City
0	Alice	25	New York
2	Charlie	35	London

3. Selecting rows and columns by label

You can specify both row and column labels to select specific entries.

```
# Select specific row and column
```

```
print(df.loc[0, 'Name'])
```

output:

Alice

4. Selecting multiple rows and columns

You can select multiple rows and columns by passing lists of labels.

```
# Select specific rows and columns
```

```
print(df.loc[[0, 2], ['Name', 'City']])
```

output:

	Name	City
0	Alice	New York

2 Charlie London

5. Selecting rows using a condition

You can apply conditions to filter rows.

```
# Select rows where age is greater than 30
```

```
print(df.loc[df['Age'] > 30])
```

output:

	Name	Age	City
2	Charlie	35	London
3	David	40	Berlin

6. Selecting specific columns

You can use `loc[]` to select all rows but only specific columns

```
# Select all rows but only specific columns
```

```
print(df.loc[:, ['Name', 'City']])
```

output:

	Name	City
0	Alice	New York
1	Bob	Paris
2	Charlie	London
3	David	Berlin

7. Setting values using loc[]

You can use `loc[]` to modify DataFrame values.

```
# Update the value at a specific location
```

```
df.loc[1, 'Age'] = 31
```

```
print(df)
```

output:

	Name	Age	City
0	Alice	25	New York
1	Bob	31	Paris
2	Charlie	35	London
3	David	40	Berlin

9. Selecting a range of rows using index labels

You can select a range of rows by specifying the start and end index labels (inclusive).

Select rows between index labels 0 and 2 (inclusive)

```
print(df.loc[0:2])
```

output:

	Name	Age	City
0	Alice	25	New York
1	Bob	31	Paris
2	Charlie	35	London

SORTING IN DATAFRAME

1. sort_values()

This method is used to sort a DataFrame by one or more columns. It can be used for sorting by ascending or descending order.

Syntax:

```
DataFrame.sort_values(by, axis=0, ascending=True, inplace=False)
```

- **by:** Column(s) or row(s) to sort by.
- **axis:** Axis to sort (0 for index/rows, 1 for columns).
- **ascending:** If True (default), sorts in ascending order and if false, means sort in descending order.
- **inplace:** If True, modifies the original DataFrame.

Example:

```
import pandas as pd

# Creating a DataFrame

data = {'Name': ['John', 'Anna', 'Peter', 'Linda', 'James'],
        'Age': [28, 22, 35, 32, 29]}

df = pd.DataFrame(data)

# Sort by Age in ascending order

df_sorted = df.sort_values(by='Age')

print(df_sorted)
```

output:

	Name	Age
1	Anna	22
0	John	28
4	James	29
3	Linda	32
2	Peter	35

2. sort_index()

This method sorts the DataFrame by its index (row or column labels)

Syntax:

```
DataFrame.sort_index(axis=0, level=None, ascending=True, inplace=False)
```

3. sort_values() with multiple columns

You can sort by multiple columns by passing a list to the by parameter.

Example:

```
# Sort by Age, then by Name

df_sorted_multi = df.sort_values(by=['Age', 'Name'], ascending=[True, True])
```

```
print(df_sorted_multi)
```

output:

	Name	Age
1	Anna	22
0	John	28
4	James	29
3	Linda	32
2	Peter	35

Isin()

The `isin()` method in pandas is used to filter data by checking if elements in a DataFrame or Series are contained within a given set of values. It returns a boolean DataFrame/Series, where each entry is either True or False depending on whether the element is present in the provided list, set, or iterable.

Syntax:

```
DataFrame.isin(values)
```

values: Can be a list, tuple, set, or dictionary of values to check against the DataFrame's elements.

Example 1: `isin()` with a Single Column

Let's say we want to check if values in a specific column belong to a list of values.

```
import pandas as pd
```

```
# Creating a DataFrame
```

```
data = {'Name': ['John', 'Anna', 'Peter', 'Linda', 'James'],  
        'Age': [28, 22, 35, 32, 29]}
```

```
df = pd.DataFrame(data)
```

```
# Check if 'Age' values are in [22, 29, 35]
```

```
result = df['Age'].isin([22, 29, 35])
```



```
print(result)
```

output:

```
0  False
1   True
2   True
3  False
4   True
```

Name: Age, dtype: bool

Example 2: Filtering Rows using isin()

You can use `isin()` to filter rows in a DataFrame that meet certain criteria.

```
# Filter rows where 'Age' is in [22, 29, 35]
filtered_df = df[df['Age'].isin([22, 29, 35])]
print(filtered_df)
```

output:

```
   Name  Age
1  Anna   22
2  Peter  35
4  James  29
```

Example 3: Using isin() with Multiple Columns and a Dictionary

You can also pass a dictionary to `isin()` to check membership for different columns with different sets of values.

```
# Check if 'Name' is in ['John', 'Anna'] and 'Age' is in [28, 29]
result = df.isin({'Name': ['John', 'Anna'], 'Age': [28, 29]})
print(result)
```

output:

	Name	Age
0	True	True
1	True	False
2	False	False
3	False	False
4	False	True

Example 4: Combining isin() with Other Conditions

You can combine isin() with other conditions for more complex filtering.

```
# Filter rows where 'Age' is in [22, 29] and 'Name' starts with 'J'
filtered_df = df[df['Age'].isin([22, 29]) & df['Name'].str.startswith('J')]
print(filtered_df)
```

output:

	Name	Age
4	James	29

SUM()

The sum() function in pandas is used to calculate the sum of values along a specified axis (rows or columns) of a DataFrame or Series.

Syntax:

```
DataFrame.sum(axis=0, skipna=True, level=None, numeric_only=None, min_count=0)
```

❓ **axis:** Defines the axis along which to sum:

- 0 (default): Sum along columns (i.e., sum of values for each column).
- 1: Sum along rows (i.e., sum of values for each row).

❓ **skipna:** Whether to skip NaN (missing) values. Default is True.

❓ **level:** For DataFrames with a MultiIndex, it sums along a particular level.

- 🔍 **numeric_only**: If True, only numeric columns are summed. Non-numeric columns are ignored.
- 🔍 **min_count**: The minimum number of valid (non-NaN) entries required to perform the sum. If fewer than min_count valid entries are present, the result is NaN.

QUERY()

The query() function in pandas provides a way to filter DataFrames using a string expression. It allows you to specify conditions directly as a string, which is then evaluated to filter rows in the DataFrame.

Syntax:

```
DataFrame.query(expr, inplace=False, **kwargs)
```

- 🔍 **expr**: A string expression representing the query to evaluate.
- 🔍 **inplace**: If True, modifies the original DataFrame instead of returning a new one.
- 🔍 **kwargs**: Additional keyword arguments, including local variables that can be used inside the query.

Example 1: Basic Filtering with query()

You can filter rows based on a condition, such as values in a column being greater than a certain threshold.

```
import pandas as pd

# Creating a DataFrame
data = {'Name': ['John', 'Anna', 'Peter', 'Linda', 'James'],
        'Age': [28, 22, 35, 32, 29],
        'City': ['NY', 'LA', 'NY', 'SF', 'LA']}

df = pd.DataFrame(data)

# Use query to filter rows where Age is greater than 30
result = df.query('Age > 30')

print(result)
```

output:

```
   Name  Age City
2 Peter  35  NY
```

3 Linda 32 SF

Explanation: The rows where Age is greater than 30 are selected

Example 2: Using Multiple Conditions

You can use logical operators like and (&), or (|), and not (~) to create more complex queries

```
# Filter rows where Age is greater than 25 and City is 'NY'
```

```
result = df.query('Age > 25 and City == "NY"')
```

```
print(result)
```

output:

	Name	Age	City
0	John	28	NY
2	Peter	35	NY

Explanation: Rows where both conditions (Age > 25 and City == "NY") are true are selected.

Example 3: Filtering Rows with Special Column Names

If the column names contain spaces or special characters, you can surround them with backticks (`column name`).

```
# DataFrame with spaces in column names
```

```
df_special = pd.DataFrame({'First Name': ['John', 'Anna', 'Peter'],  
                           'Last Name': ['Smith', 'Doe', 'Jones'],  
                           'Age': [28, 22, 35]})
```

```
# Use backticks to refer to column names with spaces
```

```
result = df_special.query("`First Name` == \"John\"")  
print(result)
```

output:

	First Name	Last Name	Age
0	John	Smith	28

Example 4: Using query() with Boolean Columns

If a column contains boolean values, you can use it directly in the query expression.

Adding a boolean column

```
df_bool = pd.DataFrame({'Name': ['John', 'Anna', 'Peter'],  
                        'Age': [28, 22, 35],  
                        'Is_Student': [True, False, True]})
```

Use query to filter rows where Is_Student is True

```
result = df_bool.query('Is_Student == True')  
print(result)
```

output:

	Name	Age	Is_Student
0	John	28	True
2	Peter	35	True

Explanation: The query filters rows where the Is_Student column is True.

Example 5: Chaining query() with Other Pandas Operations

You can chain query() with other pandas methods for more complex data manipulations.

Chain query with other operations

```
result = df.query('Age > 25').sort_values(by='Age')  
print(result)
```

output:

	Name	Age	City
0	John	28	NY

```
4 James 29 LA
3 Linda 32 SF
2 Peter 35 NY
```

Explanation: First, rows where Age > 25 are selected, and then they are sorted by Age.

PIVOT()

The pivot() function in pandas is used to reshape data in a DataFrame by converting unique values from one column into multiple columns, effectively transforming the DataFrame layout. This is particularly useful when you want to summarize data or perform aggregations.

Syntax:

```
DataFrame.pivot(index=None, columns=None, values=None)
```

❓ **index:** Column(s) to use to make new frame's index. If none, uses existing index/column.

❓ **columns:** Column(s) to use to make new frame's columns.

❓ **values:** Column(s) to use for populating new frame's values. If not specified, all remaining columns will be used.

Example 1: Basic Pivot

Let's create a simple DataFrame and see how to use pivot() to reshape it.

```
import pandas as pd
```

```
# Create a sample DataFrame
```

```
data = {
    'date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02'],
    'city': ['New York', 'Los Angeles', 'New York', 'Los Angeles'],
    'temperature': [30, 65, 28, 66]
}

df = pd.DataFrame(data)
```

```
print("Original DataFrame:")
```

```
print(df)
```

output:

	date	city	temperature
0	2023-01-01	New York	30
1	2023-01-01	Los Angeles	65
2	2023-01-02	New York	28
3	2023-01-02	Los Angeles	66

Now, let's pivot this DataFrame to have dates as the index and cities as columns.

```
# Pivot the DataFrame
```

```
pivot_df = df.pivot(index='date', columns='city', values='temperature')
```

```
print("\nPivoted DataFrame:")
```

```
print(pivot_df)
```

output:

city	Los Angeles	New York
date		
2023-01-01	65	30
2023-01-02	66	28

example 2: Handling Duplicate Entries

If you have duplicate entries for the combination of index and columns, `pivot()` will raise an error. Instead, you can use `pivot_table()` which allows for aggregation.

```
# Create a sample DataFrame with duplicate entries
```

```
data_dup = {  
    'date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02', '2023-01-01'],  
    'city': ['New York', 'Los Angeles', 'New York', 'Los Angeles', 'New York'],
```

```

    'temperature': [30, 65, 28, 66, 32]
}
df_dup = pd.DataFrame(data_dup)
print("\nDataFrame with Duplicates:")
print(df_dup)

# Using pivot_table to handle duplicates
pivot_table_df = df_dup.pivot_table(index='date', columns='city', values='temperature', aggfunc='mean')
print("\nPivot Table:")
print(pivot_table_df)

```

output:

DataFrame with Duplicates:

	date	city	temperature
0	2023-01-01	New York	30
1	2023-01-01	Los Angeles	65
2	2023-01-02	New York	28
3	2023-01-02	Los Angeles	66
4	2023-01-01	New York	32

Pivot Table:

city	Los Angeles	New York
date		
2023-01-01	65	31.0
2023-01-02	66	28.0

IsNull().sum()

It is used to calculate total number of null values present in each column.

Syntax:


```
a.isnull.sum()
```

astype()

This method allows you to convert a column (or multiple columns) to a specific datatype.

Syntax:

```
df['column_name'] = df['column_name'].astype(new_dtype)
```

Handling Invalid Data During Conversion

Sometimes, a column may have values that cannot be converted to the desired datatype. You can handle these situations using the errors parameter in `astype()`. The options are:

- `errors='raise'` (default): Throws an error if conversion fails.
- `errors='ignore'`: Leaves the column unchanged if conversion fails.
- `errors='coerce'`: Converts invalid values to NaN.

Example: Handling Invalid Values with `errors='coerce'`

```
# Sample DataFrame with mixed values
```

```
data = {'A': ['1', '2', 'invalid', '4']}
```

```
df = pd.DataFrame(data)
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Convert column 'A' to numeric, invalid values become NaN
```

```
df['A'] = pd.to_numeric(df['A'], errors='coerce')
```

```
print("\nDataFrame after attempting to convert 'A' to numeric:")
```

```
print(df)
```

output:

Original DataFrame:

```
      A
0     1
1     2
2  invalid
3     4
```

DataFrame after attempting to convert 'A' to numeric:

```
      A
0  1.0
1  2.0
2  NaN
3  4.0
```

Application: Use errors='coerce' to convert non-convertible values into NaN when changing column datatypes.

Isna()

In Pandas, `isna()` (or its equivalent `isnull()`) is used to detect missing or NaN (Not a Number) values in a DataFrame or Series. The function returns a DataFrame or Series of the same shape as the input but with Boolean values: True where the element is NaN and False otherwise.

Example1: Basic Usage to Detect Missing Values

The primary use of `isna()` is to detect NaN values in a DataFrame or Series.

```
import pandas as pd
```

```
import numpy as np
```

```
# Creating a DataFrame with some NaN values
```

```
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
```

```
'B': [5, np.nan, np.nan, 8],  
'C': ['a', 'b', 'c', np.nan]  
})
```

Detecting NaN values

```
na_mask = df.isna()
```

```
print(na_mask)
```

output:

	A	B	C
0	False	False	False
1	False	True	False
2	True	True	False
3	False	False	True

Example2: Summing Missing Values per Column

You can use `isna()` in combination with the `sum()` method to count the number of missing values in each column.

Count of NaN values per column

```
missing_per_column = df.isna().sum()
```

```
print(missing_per_column)
```

output:

A 1

B 2

C 1

dtype: int64

fillna()

example1. Filling Missing Values with a Scalar Value

This is the most common use case, where we replace all NaN values with a fixed value.

```
import pandas as pd
import numpy as np

# Creating a DataFrame with NaN values
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [np.nan, 5, np.nan, 8],
    'C': ['a', np.nan, 'c', 'd']
})

# Fill all NaN values with 0
df_filled = df.fillna(0)
print(df_filled)
```

output:

	A	B	C
0	1.0	0.0	a
1	2.0	5.0	0
2	0.0	0.0	c
3	4.0	8.0	d

example2: Filling with Different Values for Each Column

You can pass a dictionary to fillna() to fill each column with a different value.

```
# Fill NaN in column 'A' with 100, and NaN in column 'B' with 200
df_filled = df.fillna({'A': 100, 'B': 200})
```

```
print(df_filled)
```

output:

	A	B	C
0	1.0	200.0	a
1	2.0	5.0	NaN
2	100.0	200.0	c
3	4.0	8.0	d

Example 3: want to fill missing values into columns containing values in the form of float or integer with mean value of each respective column , without changing the datatype of columns

Example Code:

python

```
import pandas as pd

# Step 1: Read the CSV file into a DataFrame
df = pd.read_csv('your_file.csv')

# Step 2: Select only numeric columns (float or int)
numeric_cols = df.select_dtypes(include=['float64', 'int64'])

# Step 3: Fill missing values (NaN) in numeric columns with the mean of
df[numeric_cols.columns] = numeric_cols.fillna(numeric_cols.mean())

# Step 4: Optionally, save the modified DataFrame back to a CSV file
df.to_csv('filled_file.csv', index=False)

# Print the result for verification
print(df)
```

Replace()

It can be used to replace specific values, multiple values, or even entire patterns.

Syntax:

```
df.replace(to_replace, value, inplace=False, limit=None, regex=False, method=None)
```

🔍 to_replace: The value(s) you want to replace. Can be a single value, list, dictionary, or regex.

value: The value(s) to replace the old values with.

Example1: Replacing Values Using a Dictionary

You can use a dictionary to replace values where the keys are the old values and the values are the new ones.

```
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [5, 6, 7, 8, 9]
})

# Replace using a dictionary
df_replaced_dict = df.replace({'A': {1: 100, 3: 300}, 'B': {6: 60}})
print("\nDataFrame after replacing values using a dictionary:")
print(df_replaced_dict)
```

output:

DataFrame after replacing values using a dictionary:

	A	B
0	100	5.0
1	2	60.0
2	300	7.0
3	4	8.0
4	5	9.0

Example2: Replacing Values Based on a Condition

You can use `replace()` to perform conditional replacement, similar to using `np.where()`.

Replace Negative Values with NaN

```
import numpy as np
```

```
# Sample DataFrame with negative values
```

```
df_negative = pd.DataFrame({  
    'A': [1, -2, 3, -4, 5],  
    'B': [5, -6, 7, 8, -9]  
})
```

```
# Replace negative values with NaN
```

```
df_replaced_condition = df_negative.replace(to_replace=[-2, -4, -6, -9], value=np.nan)  
print("\nDataFrame after replacing negative values with NaN:")  
print(df_replaced_condition)
```

output:

DataFrame after replacing negative values with NaN:

	A	B
0	1.0	5.0
1	NaN	NaN
2	3.0	7.0
3	NaN	8.0
4	5.0	NaN

Example3. Replacing Entire Rows or Columns

You can replace entire rows or columns with specific values if they meet a condition or contain certain values.

```
# Replace all values in column 'B' with 0
```

```
df_replaced_col = df.replace({'B': df['B'].tolist()}, 0)  
print("\nDataFrame after replacing all values in column 'B' with 0:")  
print(df_replaced_col)
```


Example4: imagine we have a csv file which includes a column named as gender and that column only contain two string values as male and female . write a code to change the string male to value 0 and string female to value 1 in pandas.

```
import pandas as pd
```

```
# Sample DataFrame
```

```
data = {'gender': ['male', 'female', 'male', 'female', 'male']}
```

```
df = pd.DataFrame(data)
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Replace 'male' with 0 and 'female' with 1 in the 'gender' column
```

```
df['gender'] = df['gender'].replace({'male': 0, 'female': 1})
```

```
print("\nDataFrame after replacement:")
```

```
print(df)
```

output:

Original DataFrame:

```
gender
0    male
1  female
2    male
3  female
4    male
```

DataFrame after replacement:

```
gender
```

0 0

1 1

2 0

3 1

4 0

MERGE()

merge() function is a powerful tool for combining two or more DataFrames based on a common column or index. The merge() function has several use cases and parameters that allow you to control how the data is combined.

Here's a detailed breakdown of the most common use cases of merge() in pandas:

1. Basic Merge on a Single Key Column

This is the simplest form of merging, where you combine two DataFrames based on a common column.

Example:

```
import pandas as pd

# Creating two dataframes
df1 = pd.DataFrame({
    'ID': [1, 2, 3, 4],
    'Name': ['Alice', 'Bob', 'Charlie', 'David']
})

df2 = pd.DataFrame({
    'ID': [3, 4, 5, 6],
    'Age': [23, 30, 22, 25]
})

# Merging on 'ID' column
merged_df = pd.merge(df1, df2, on='ID')
print(merged_df)
```

Output:

	ID	Name	Age
0	3	Charlie	23
1	4	David	30

Explanation: This merges the two DataFrames where the values of the 'ID' column match.

2. Merge with Different Column Names

If the columns you want to merge on have different names in the two DataFrames, you can specify them using the left_on and right_on parameters.

Example:

```
# DataFrames with different column names for the key
```

```
df1 = pd.DataFrame({  
    'Student_ID': [1, 2, 3, 4],  
    'Name': ['Alice', 'Bob', 'Charlie', 'David']  
})
```

```
df2 = pd.DataFrame({  
    'ID': [3, 4, 5, 6],  
    'Age': [23, 30, 22, 25]  
})
```

```
# Merging on different column names
```

```
merged_df = pd.merge(df1, df2, left_on='Student_ID', right_on='ID')  
print(merged_df)
```

Output:

	Student_ID	Name	ID	Age
0	3	Charlie	3	23
1	4	David	4	30

3. Merge with Different Types of Joins

By default, `merge()` performs an **inner join**, but you can change the type of join using the `how` parameter. There are several options for the `how` parameter:

- 'inner' (default) – Returns only the rows with matching keys in both DataFrames.
- 'left' – Returns all rows from the left DataFrame, and matching rows from the right DataFrame.
- 'right' – Returns all rows from the right DataFrame, and matching rows from the left DataFrame.
- 'outer' – Returns all rows from both DataFrames, filling in missing values with NaN where there's no match.

Example (Inner Join):

```
merged_df = pd.merge(df1, df2, on='ID', how='inner')  
print(merged_df)
```

Output:

```
ID  Name Age
0  3  Charlie  23
1  4   David  30
```

Example (Left Join):

```
merged_df = pd.merge(df1, df2, on='ID', how='left')
print(merged_df)
```

Output:

```
ID  Name Age
0  1  Alice NaN
1  2   Bob NaN
2  3  Charlie 23.0
3  4   David 30.0
```

Example (Right Join):

```
merged_df = pd.merge(df1, df2, on='ID', how='right')
print(merged_df)
```

Output:

```
ID  Name Age
0  3  Charlie  23
1  4   David  30
2  5   NaN   22
3  6   NaN   25
```

Example (Outer Join):

```
merged_df = pd.merge(df1, df2, on='ID', how='outer')
print(merged_df)
```

Output:

```
ID  Name Age
0  1  Alice NaN
1  2   Bob NaN
2  3  Charlie 23.0
3  4   David 30.0
4  5   NaN  22.0
```

5 6 NaN 25.0

4. Merging on Index

You can merge DataFrames using their index by setting `left_index=True` and `right_index=True`.

Example:

```
df1 = pd.DataFrame({  
    'Name': ['Alice', 'Bob', 'Charlie', 'David']  
}, index=[1, 2, 3, 4])
```

```
df2 = pd.DataFrame({  
    'Age': [23, 30, 22, 25]  
}, index=[3, 4, 5, 6])
```

```
# Merging using index
```

```
merged_df = pd.merge(df1, df2, left_index=True, right_index=True)  
print(merged_df)
```

Output:

```
   Name  Age  
3  Charlie  22  
4   David  25
```

CONCAT()

The `concat()` function is used to **concatenate or stack DataFrames either vertically (row-wise) or horizontally (column-wise)**. It simply concatenates data without the need for matching on a specific key. It's more about stacking DataFrames and doesn't require any common columns or keys.

Syntax of `concat()`

```
# axis=0 for vertical, axis=1 for horizontal  
pd.concat([df1, df2], axis=0)
```

Concatenating with Different Indexes

If the DataFrames have different indexes, `concat()` will align them based on the index, and you'll get NaN values where the indexes do not match.

Example:

```
df1 = pd.DataFrame({  
    'ID': [1, 2],  
    'Name': ['Alice', 'Bob']  
}, index=[0, 1])
```

```
df2 = pd.DataFrame({  
    'ID': [3, 4],  
    'Name': ['Charlie', 'David']  
}, index=[2, 3])
```

```
# Concatenating along rows with different indexes
```

```
result = pd.concat([df1, df2])  
print(result)
```

Output:

	ID	Name
0	1	Alice
1	2	Bob
2	3	Charlie
3	4	David

Explanation: The rows from both DataFrames are appended, and their indexes are preserved. Here, since both DataFrames have different indices, pandas handles the alignment.

Resetting Index After Concatenation

You can reset the index after concatenating with the `ignore_index=True` parameter to avoid having duplicate or mismatched indices.

Example:

```
result = pd.concat([df1, df2], ignore_index=True)
```

```
print(result)
```

Output:

	ID	Name
0	1	Alice
1	2	Bob
2	3	Charlie
3	4	David

Explanation: The index is reset to a new range (0 to n-1), and the original indices are discarded.